

## Pepfar Project – Github Basics

Repo: <https://github.com/akilansankaran/pepfar-project>

### Setup

1. Accept the GitHub invite. This invite lies in your email or at [github.com/notifications](https://github.com/notifications). You must accept before pushing any code or pulling any code in this situation, since otherwise it will not let you.

2. Install Git from <https://git-scm.com>. On a Mac you can run

**xcode-select --install instead.**

Let me know if you have any issues with this!

3. Set your name and email (if you have not set this up previously)

Note that every commit you make is stamped with this name and email.

Use the email on your GitHub account so commits link to your profile. **In th terminal**, run the following commands:

**git config --global user.name "Your Name"**

**git config --global user.email "you@example.com"**

4. Log in to GitHub:

- Install the GitHub CLI from <https://cli.github.com> first.
- Choose GitHub.com, then HTTPS, then "Login with a web browser".
- This saves your login so git push and pull work in this sense from your browser.
- Then, **in the terminal**, run

**gh auth login**

5. Download the repo. You'll now be copying the entire project and the history of the project into a new folder called pepfar-project, inside the folder you're in now. Thus, the contents of this folder will be precisely the ones that are pulled from the **remote** version at the time that you "cloned" it into your own computer. Thus, you should run:

**git clone https://github.com/akilansankaran/pepfar-project.git**

Then, move into the project. Run all later git commands from in here.

**cd pepfar-project**

**Every time you work**, follow the following steps.

- Begin by downloading and merging everyone else's latest changes into your copy. This should be done before you start, so that you build on the newest version (note that you might have conflicting files here! Let's hope not, but you will have to go through these – or have Claude do it – and make sure that they do not conflict).
- Therefore, before you start, it's useful to run

**git pull**

or `git pull --no-rebase main` if you would like to maintain those changes on main.

You then edit files as you do your amazing Eclabs work! After making these changes:

List the files you've changed or added since your last commit using the following command:

(**Red** = changed but not staged; **green** = staged and ready to commit.)

**git status**

Now, you can choose which changes + files you want to go into your commit. You can name specific files (i.e. `git add Akilan.py`) or you can `git add .` to **add all of your changes** (be careful with this!). For instance, you can add a file like

**git add analysis/replication/did\_baseline.R**

Then, you can **commit** those files locally in your copy, which thus saves a snapshot of the staged changes in your local copy. The message should say what changed in this case and how, for instance "Add HIPC controls to baseline DiD". This commit message looks like:

**git commit -m "What you changed"**

Then, upload your commits to GitHub so everyone else can pull them.

Until you push, your commits exist only on your computer.

**git push**

**And that's it!** Some technicalities below, but happy git-ing!

### **If a push is rejected....**

“rejected” or “fetch first” means someone pushed while you were working.

Pull their changes in first; usually git merges them automatically, by:

```
git pull
```

If git reports a CONFLICT, open the listed files. Keep the correct lines between the <<<<<<< and >>>>>>> markers, delete the markers, then stage the fixed files and commit:

```
git add <fixed file>
```

```
git commit -m "Merge latest changes"
```

Now the push will go through, so run:

```
git push
```

### **Other useful commands ...**

This one throws away your uncommitted edits to one file, back to the last commit.

This can't be undone, so check first.

```
git restore <file>
```

This one shows recent commits: who made each one, when, and the message.

```
git log --oneline
```

**Be careful about committing data files!**

Raw and restricted data (IHME, Afrobarometer) go in the shared dropbox/drive/etc., consult with your PL hence. The .gitignore already blocks most of them, but check git status before you commit.

If you have any issues, ask Emma or Akilan or Claude ☺